

Desafío 2

R.A.T Industries

Integrantes: Pablo Varetto D.
Joaquin Poblete
Felicie Nguyen
Enrique Rebolledo C.
Marina Olmedo G.

Profesor: Harold Valenzuela Coloma

Auxiliar: Fernando Navarrete U.

Ayudantes: Valentina Abarca G
Fernanda Echeverría Rojas

Fecha de entrega: 03 de Noviembre de 2025
Santiago de Chile

Índice de Contenidos

1. Diagrama del sistema	1
2. Introducción	2
3. Colores a Clasificar	2
4. Calibración del Sensor TCS3200	2
5. Problemas Encontrados	2
6. Códigos Implementados	2
6.1. Código de Calibración	3
6.2. Código de Funcionamiento	4
7. Conclusión	9

Índice de Figuras

1.1. Diagrama de conexiones del sistema, el que incluye los servomotores, el sensor TCS3200, el motor dc y el arduino UNO además del driver L298N.	1
--	---

Índice de Tablas

4.1. Valores RGB obtenidos en calibración	2
---	---

Índice de Códigos

1. Código de calibración del sensor TCS3200	3
2. Código principal del sistema de clasificación	4

1. Diagrama del sistema

Se presenta el dibujo de la arquitectura.

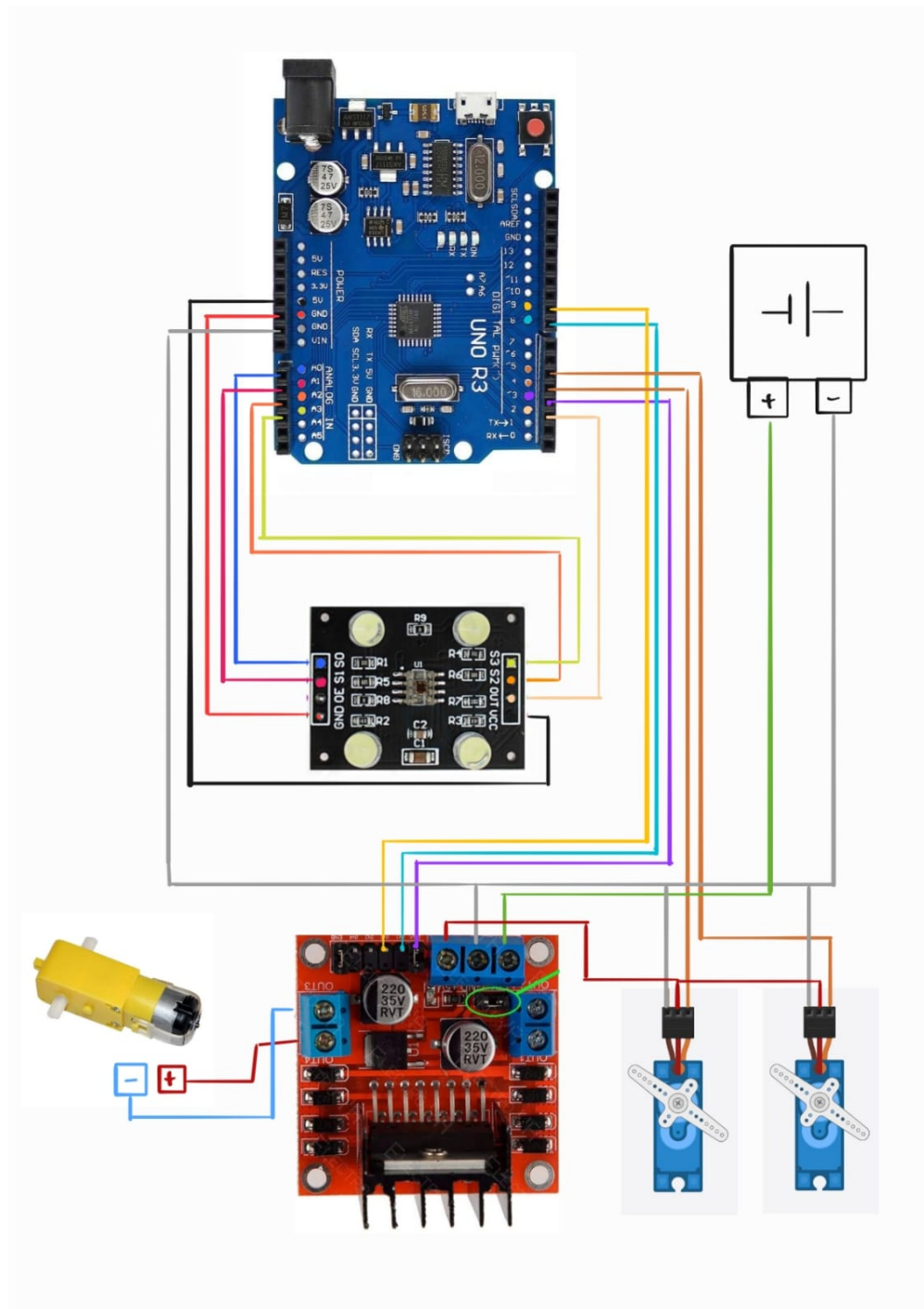


Figura 1.1: Diagrama de conexiones del sistema, el que incluye los servomotores, el sensor TCS3200, el motor dc y el arduino UNO además del driver L298N.

2. Introducción

Se presenta el informe técnico del Desafío 2 de R.A.T. Industries, cuyo objetivo consiste en clasificar automáticamente los R.E.P.O.S según su color utilizando el sensor TCS3200 y un sistema de correa transportadora controlado por servomotores.

3. Colores a Clasificar

Los colores entregados de R.E.P.O.S para clasificación fueron:

- Rojo
- Morado
- Blanco

4. Calibración del Sensor TCS3200

Se realizó la calibración del sensor TCS3200 siguiendo el procedimiento establecido. Los valores RGB obtenidos durante la calibración fueron:

Tabla 4.1: Valores RGB obtenidos en calibración

Color	R (Rojo)	G (Verde)	B (Azul)
Blanco	35	34	30
Negro	174	166	132
Rojo	150	50	60
Morado	165	150	170

5. Problemas Encontrados

Durante la implementación del sistema se presentó el siguiente inconveniente:

- La correa transportadora posiblemente quedó sin batería, lo que afectó su funcionamiento durante las pruebas.

6. Códigos Implementados

6.1. Código de Calibración

Código 1: Código de calibración del sensor TCS3200

```

1  // /*
2  Calibración Sensor de Color TCS3200 / GY-31 (pines analógicos)
3  Conexiones:
4  S0 -> A0
5  S1 -> A1
6  S2 -> A2
7  S3 -> A3
8  OUT -> D2
9  VCC -> 5V
10 GND -> GND
11 */
12
13 #define S0 A0      // Control frecuencia de salida
14 #define S1 A1      // Control frecuencia de salida
15 #define S2 A2      // Selección de filtro de color (R/G/B)
16 #define S3 A3      // Selección de filtro de color (R/G/B)
17 #define sensorOut 2 // Pin de salida del sensor (digital)
18
19 int redFreq, greenFreq, blueFreq;
20
21 // Lee un color con opción de promediar varias lecturas para estabilidad
22 unsigned long leerColor(bool s2State, bool s3State, uint8_t muestras = 3) {
23     digitalWrite(S2, s2State);
24     digitalWrite(S3, s3State);
25
26     // Pequeño tiempo para que el filtro cambie establemente
27     delayMicroseconds(300);
28
29     unsigned long acc = 0;
30     for (uint8_t i = 0; i < muestras; i++) {
31         // pulseIn mide duración del pulso LOW (usamos timeout para evitar bloqueos)
32         unsigned long t = pulseIn(sensorOut, LOW, 25000UL); // 25 ms timeout
33         if (t == 0) {
34             // Si no hubo pulso a tiempo, intenta una vez más
35             t = pulseIn(sensorOut, LOW, 25000UL);
36         }
37         acc += (t > 0 ? t : 0);
38     }
39     return (muestras ? acc / muestras : acc);
40 }
41
42 void setup() {
43     Serial.begin(9600);
44
45     pinMode(S0, OUTPUT);
46     pinMode(S1, OUTPUT);
47     pinMode(S2, OUTPUT);

```

```

48  pinMode(S3, OUTPUT);
49  pinMode(sensorOut, INPUT);
50
51  // Escala al 100 % para máxima sensibilidad
52  digitalWrite(S0, HIGH);
53  digitalWrite(S1, HIGH);
54
55  Serial.println("Calibración de sensor de color (GY-31) iniciada...");
56  Serial.println("Coloca NEGRO y luego BLANCO para registrar R, G, B mínimos y máximos.");
57  }
58
59  void loop() {
60    // Filtro ROJO (S2=LOW, S3=LOW)
61    redFreq = leerColor(LOW, LOW);
62
63    // Filtro VERDE (S2=HIGH, S3=HIGH)
64    greenFreq = leerColor(HIGH, HIGH);
65
66    // Filtro AZUL (S2=LOW, S3=HIGH)
67    blueFreq = leerColor(LOW, HIGH);
68
69    Serial.print("R: "); Serial.print(redFreq);
70    Serial.print(" G: "); Serial.print(greenFreq);
71    Serial.print(" B: "); Serial.println(blueFreq);
72
73    delay(500); }

```

6.2. Código de Funcionamiento

Código 2: Código principal del sistema de clasificación

```

1  // #include <Servo.h>
2
3  /*
4   Conexiones:
5   - TCS3200: S0->A0, S1->A1, S2->A2, S3->A3, OUT->D2, VCC->5V, GND->GND
6   - L298N: ENA=3 (PWM), IN1=8, IN2=9
7   - Servos: servo1=D5, servo2=D4
8  */
9
10 // ===== L298N =====
11 const int ENA = 3;
12 const int IN1 = 8;
13 const int IN2 = 9;
14
15 // ===== Servos =====
16 Servo servo1; // D5
17 Servo servo2; // D4
18

```

```

19 // ===== GY-31 (TCS3200) =====
20 const int S0 = A0;
21 const int S1 = A1;
22 const int S2 = A2;
23 const int S3 = A3;
24 const int SENSOR_OUT = 2; // salida de frecuencia del sensor
25
26 // ===== Calibración (TODO: Pegar tus valores medidos) =====
27 int R_BLACK = 174, G_BLACK = 166, B_BLACK = 132; // <-- "NEGRO" (R,G,B)
28 int R_WHITE = 35, G_WHITE = 34, B_WHITE = 30; // <-- "BLANCO" (R,G,B)
29
30 // ===== Parámetros de operación =====
31 const uint8_t SERVO_NEUTRO = 90;
32 const int PWM_CINTA = 100; // velocidad cinta (0-255)
33 const unsigned CINTA_MS = 10000; // cuánto avanza la cinta por ciclo (ms)
34 const unsigned ESPERA_OBJ_MS = 2000; // "simula" que ponen objeto
35
36 // Umbral de clasificación (0-255 en espacio normalizado luminosidad)
37 uint8_t T_dark = 25; // negro (muy oscuro) <-- puedes ajustar
38 uint8_t T_white = 200; // blanco (muy claro) <-- puedes ajustar
39 uint8_t T_color = 15; // diferencia mínima entre canal ganador y los demás
40
41 // ===== Helpers del motor =====
42 void motorStart(int pwm) {
43   digitalWrite(IN1, HIGH);
44   digitalWrite(IN2, LOW);
45   analogWrite(ENA, pwm);
46 }
47 void motorStop() {
48   analogWrite(ENA, 0);
49   digitalWrite(IN1, LOW);
50   digitalWrite(IN2, LOW);
51 }
52
53 // ===== Lectura del TCS3200 con pulseIn =====
54 unsigned long leerColor(bool s2, bool s3, uint8_t muestras = 4) {
55   digitalWrite(S2, s2);
56   digitalWrite(S3, s3);
57   delayMicroseconds(300); // tiempo de asentamiento del filtro
58
59   unsigned long acc = 0;
60   for (uint8_t i = 0; i < muestras; i++) {
61     unsigned long t = pulseIn(SENSOR_OUT, LOW, 25000UL); // timeout 25 ms
62     if (t == 0) t = pulseIn(SENSOR_OUT, LOW, 25000UL); // intento extra si hubo timeout
63     acc += t;
64   }
65   return (muestras ? acc / muestras : acc);
66 }
67
68 // Mapea tiempo de pulso a intensidad 0-255 (invertido) con límites
69 int mapInvertidoConstrain(int t, int t_black, int t_white) {

```

```

70  if (t_black == t_white) return 0;
71  long val = map((long)t, (long)t_black, (long)t_white, 0, 255);
72  return (int)constrain(val, 0, 255);
73 }
74
75 // Clasificación devuelve etiqueta y "acción" (1 o 2) que activará luego los servos
76 const char* clasificarRGB(int R, int G, int B, int &accion) {
77     accion = 0; // 0 = sin servo; 1 = acción 1; 2 = acción 2
78
79     // Chequeo de extremos por luminancia aproximada
80     int Y = (R*30 + G*59 + B*11) / 100;
81
82     // ===== EJEMPLOS ya implementados =====
83     // NEGRO: muy bajo en los tres canales
84     if (Y < T_dark) {
85         // Ejemplo: NEGRO -> SIN ACCIÓN (solo cinta)
86         accion = 0;
87         return "NEGRO";
88     }
89
90
91     if (Y > T_white) { // Blanco sigue derecho
92         accion = 3;
93         return "BLANCO";
94     }
95
96     // TODO(Alumno): ROJO
97     // R dominante sobre G y B
98     if (R > G + T_color && R > B + T_color) {
99         accion = 1; // Rojo -> ACCION 1
100         return "ROJO";
101     }
102
103     // TODO(Alumno): MORADO (violeta)
104     // R y B dominantes sobre G. Se chequea DESPUÉS de Rojo.
105     if (R > G + T_color && B > G + T_color) {
106         accion = 2; // Morado -> ACCION 2
107         return "MORADO";
108     }
109
110     // Otras posibilidades (gris, indefinido, etc.)
111     if (abs(R-G) < 20 && abs(G-B) < 20) return "GRIS";
112
113     return "INDEFINIDO";
114 }
115
116 void setup() {
117     Serial.begin(9600);
118
119     // Motor
120     pinMode(ENA, OUTPUT);

```

```

121  pinMode(IN1, OUTPUT);
122  pinMode(IN2, OUTPUT);
123  motorStop();
124
125  // Servos
126  servo1.attach(5);
127  servo2.attach(4);
128  servo1.write(SERVO_NEUTRO);
129  servo2.write(SERVO_NEUTRO);
130
131  // Sensor TCS3200
132  pinMode(S0, OUTPUT);
133  pinMode(S1, OUTPUT);
134  pinMode(S2, OUTPUT);
135  pinMode(S3, OUTPUT);
136  pinMode(SENSOR_OUT, INPUT);
137
138  // Sensibilidad: 100 % (si se satura, cambiar a S0=HIGH, S1=LOW 20 %)
139  digitalWrite(S0, HIGH);
140  digitalWrite(S1, HIGH);
141
142  Serial.println("Desafío 2 - Plataforma de Sorting por Color lista.");
143 }
144
145 void loop() {
146  // --- 1) Espera objeto (simulada) ---
147  motorStop();
148  servo1.write(SERVO_NEUTRO);
149  servo2.write(SERVO_NEUTRO);
150  Serial.println("\nEsperando objeto...");
151  delay(ESPERA_OBJ_MS);
152
153  // --- 2) Medición del color ---
154  // ROJO: S2=LOW, S3=LOW
155  unsigned long tR = leerColor(LOW, LOW);
156  // VERDE: S2=HIGH, S3=HIGH
157  unsigned long tG = leerColor(HIGH, HIGH);
158  // AZUL: S2=LOW, S3=HIGH
159  unsigned long tB = leerColor(LOW, HIGH);
160
161  // Normalización invertida (0-255: 0 oscuro, 255 claro)
162  int R = mapInvertidoConstrain(tR, R_BLACK, R_WHITE);
163  int G = mapInvertidoConstrain(tG, G_BLACK, G_WHITE);
164  int B = mapInvertidoConstrain(tB, B_BLACK, B_WHITE);
165
166  int accion = 0;
167  const char* etiqueta = clasificarRGB(R, G, B, accion);
168
169  Serial.print("tR:"); Serial.print(tR);
170  Serial.print(" tG:"); Serial.print(tG);
171  Serial.print(" tB:"); Serial.print(tB);

```

```
172 Serial.print(" | R:"); Serial.print(R);
173 Serial.print(" G:"); Serial.print(G);
174 Serial.print(" B:"); Serial.print(B);
175 Serial.print(" -> "); Serial.println(etiqueta);
176
177 // --- 3) Acción según color ---
178
179 switch (accion) {
180     case 1:
181         // Mueve el servo 1 (D5) para desviar el objeto ROJO
182         servo1.write(45); // Ajusta este ángulo (podría ser 0 o 180)
183         Serial.println("ACCION 1: Desviando ROJO con Servo 1.");
184         break;
185
186     case 2:
187         // Mueve el servo 2 (D4) para desviar el objeto MORADO
188         servo2.write(45); // Ajusta este ángulo (podría ser 0 o 180)
189         Serial.println("ACCION 2: Desviando MORADO con Servo 2.");
190         break;
191
192     case 3:
193         // Mueve ambos servos para desviar el objeto BLANCO
194         servo1.write(90);
195         servo2.write(90);
196         delay(100);
197         servo1.write(0);
198         servo2.write(0);
199         break;
200
201     default: // Acción 0 (NEGRO, BLANCO, AZUL, VERDE, etc.)
202         Serial.println("Sin acción de servo (solo cinta).");
203         break;
204 }
205
206 delay(400);
207
208 // --- 4) Mover cinta ---
209 Serial.println("Cinta en movimiento...");
210 motorStart(PWM_CINTA);
211 delay(CINTA_MS);
212 motorStop();
213
214 // --- 5) Reset servos ---
215 servo1.write(SERVO_NEUTRO);
216 servo2.write(SERVO_NEUTRO);
217
218 Serial.println("Ciclo completo.");
219 delay(1500);
220 }
```

7. Conclusión

El sistema implementado permite clasificar los R.E.P.O.S según su color mediante el sensor TCS3200. A pesar del inconveniente con la batería de la correa transportadora, se logró el correcto funcionamiento del sistema de detección y clasificación de colores.